

CAHIER DES RECETTES ET RAPPORT D'INTÉGRATION

Plateforme de Documentation Banc KNX

Nathan Bussolaro & Yanis Boussaid

Table des matières

Plateforme de Documentation Banc KNX.....	1
1. Introduction et Périmètre du Projet.....	3
2. Chronique du Développement : Méthodologie de A à Z	4
2.1. Conception de l'Architecture et Infrastructures (Docker)	4
2.2. Gestion du Réseau et Sécurité SSL (Let's Encrypt)	4
2.3. Développement Backend et Base de Données	5
2.4. Interface Utilisateur (Frontend) et Corrections UX.....	5
3. Protocole de Recettage Détaillé (Cahier de Tests).....	6
3.1. Infrastructure, Réseau et Base de Données	6
3.2. Espace Formateur et Administration.....	7
3.3. Espace Apprenti (WebAuthn et Consultation)	9
4. Bilan Global et Conclusion	11

1. Introduction et Périmètre du Projet

Ce document exhaustif retrace l'intégralité du cycle de développement, d'intégration et de validation de la plateforme de documentation pour le Banc KLX. Il ne se contente pas de lister les tests finaux, mais détaille la méthodologie, les choix architecturaux, et les résolutions de blocages techniques rencontrés tout au long du projet.

L'objectif de cette plateforme est de moderniser l'accès aux travaux pratiques (TP) pour les apprentis en introduisant une infrastructure robuste et un système d'authentification biométrique sans mot de passe (WebAuthn / Passkeys), tout en garantissant une sécurité maximale.

2. Chronique du Développement : Méthodologie de A à Z

2.1. Conception de l'Architecture et Infrastructures (Docker)

Dès le début du projet, le choix s'est porté sur une architecture en micro-services conteneurisée afin d'assurer l'isolation, la sécurité et la portabilité de l'environnement sur un serveur Ubuntu 26.04 LTS (ou 24.04 LTS). L'infrastructure repose sur Docker Compose orchestrant trois services distincts :

- **Le service Base de données (db)** : Conteneur MySQL 8.0 doté d'un volume persistant pour sauvegarder les données (utilisateurs, classes, modules, clés d'accès) et d'un script d'initialisation automatique (init.sql).
- **Le service Backend (web)** : Conteneur exécutant Python 3.10 avec le framework Flask, gérant la logique applicative, les API WebAuthn et le rendu des templates HTML (Jinja2).
- **Le service Reverse Proxy (proxy)** : Conteneur Nginx (Alpine) dont le rôle exclusif est de chiffrer le trafic en HTTPS et de rediriger les requêtes vers le backend Flask.

2.2. Gestion du Réseau et Sécurité SSL (Let's Encrypt)

L'implémentation de la technologie WebAuthn exigeant un contexte sécurisé strict, le projet a évolué d'un simple certificat auto-signé vers une véritable infrastructure d'entreprise :

- **Acquisition et routage du domaine** : Le domaine *projetdocuyanishnathan.fr* a été configuré via un enregistrement DNS de type A pour pointer vers l'IP locale du serveur (192.168.2.100).
- **Génération du certificat officiel** : Utilisation de Certbot avec la méthode de challenge DNS (enregistrement TXT) pour obtenir un certificat Let's Encrypt valide sans exposer le réseau local à internet.
- **Résolution de conflits** : Ajustement du paramètre `RP_ID` dans Flask pour correspondre au domaine officiel, levant ainsi les blocages de sécurité biométrique imposés par les navigateurs (notamment Safari/iOS).

2.3. Développement Backend et Base de Données

Le développement du serveur Flask a nécessité la gestion de plusieurs évolutions complexes :

1. **Le problème de l'œuf et de la poule** : Implémentation d'une fonction `check_db_setup()` qui vérifie dynamiquement si un mot de passe maître existe et invite l'administrateur à le configurer lors de son premier accès à `/admin/setup`.
2. **Refonte de la librairie WebAuthn** : Correction d'une erreur 500 liée à l'obsolescence de la méthode `options.json()` remplacée par `options_to_json()` pour garantir la bonne communication du challenge cryptographique avec les appareils des apprentis.
3. **Structure Relationnelle** : Mise en place de tables SQL avec clés étrangères (ON DELETE CASCADE) permettant la création de classes, l'assignation d'apprentis à ces classes, et la gestion granulaire des permissions de visibilité des modules.

2.4. Interface Utilisateur (Frontend) et Corrections UX

L'interface a été entièrement modernisée à l'aide de Bootstrap 5. Plusieurs défis techniques ont été relevés côté client :

- **Résolution des liens JSDelivr** : Remplacement des liens CDN obsolètes (unpkg) qui causaient des erreurs 404 lors du chargement de la librairie `webauthn-json`.
- **Éditeur de texte riche (Summernote)** : Intégration de l'éditeur WYSIWYG pour la création de modules (intégration de vidéos YouTube, formatage de texte).
- **Conflit de Focus Bootstrap/Summernote** : Résolution des bugs UI où Bootstrap empêchait la saisie de liens dans la modale Summernote. La solution a consisté à forcer `dialogsInBody: false` et à utiliser l'attribut `data-bs-focus="false"`.
- **Affiliation des modules** : Modification dynamique de la base de données via `ALTER TABLE` pour ajouter une colonne `linked_module_id`, accompagnée d'un badge visuel sur les cartes de modules.

3. Protocole de Recettage Détaillé (Cahier de Tests)

3.1. Infrastructure, Réseau et Base de Données

ID	Pré-requis	Description de l'action	Résultat Attendu	Statut
INF-01	Serveur Ubuntu démarré	Exécuter docker compose up -d depuis la racine du projet.	Les conteneurs klx_mysql, klx_web, et klx_proxy démarrent sans erreur.	VALIDE
INF-02	Conteneurs actifs	Saisir http://projetdocuyanishnathan.fr dans le navigateur.	Nginx redirige automatiquement vers https://... avec un code 301.	VALIDE
INF-03	Navigation HTTPS	Cliquer sur le cadenas dans la barre d'adresse de Safari ou Chrome.	Le certificat est reconnu comme valide et délivré par Let's Encrypt.	VALIDE
INF-04	Base de données vierge	Accès initial à l'application.	La fonction <code>check_db_setup()</code> crée la colonne mot de passe, et le script <code>init.sql</code> initialise toutes les tables de relations.	VALIDE

3.2. Espace Formateur et Administration

ID	Pré-requis	Description de l'action	Résultat Attendu	Statut
ADM-01	Premier lancement global	Tenter d'accéder à /admin.	Le serveur redirige vers /admin/setup pour forcer la création du mot de passe maître.	VALIDE
ADM-02	Mot de passe configuré	Saisir un mauvais mot de passe sur /admin/login.	Refus d'accès avec un message d'erreur rouge affiché via le template Bootstrap.	VALIDE
ADM-03	Connecté en Admin	Création d'un apprenti (Saisie : Prénom "Jean", Nom "Dupont").	L'application fusionne en "Jean DUPONT", sauvegarde le token et affiche le QR Code dynamique.	VALIDE
ADM-04	Connecté en Admin	Gestion des classes : Création d'une classe "BTS CIEL" et assignation d'un apprenti.	Les données sont enregistrées dans la table de liaison user_classes.	VALIDE

ID	Pré-requis	Description de l'action	Résultat Attendu	Statut
ADM-05	Connecté en Admin	Création de module avec Summernote (insertion de lien web ou vidéo).	La modale ne bloque pas le curseur (focus Bootstrap désactivé) et sauvegarde le HTML dans la colonne comment.	VALIDE
ADM-06	Module existant	Modification d'un module pour l'affilier à un module parent.	L'ID est mis à jour dans linked_module_id, et un badge cyan s'affiche sur la carte du module.	VALIDE
ADM-07	Utilisateur existant	Cliquer sur "Réinitialiser la clé" dans l'onglet des utilisateurs.	L'ancienne clé passkeys est supprimée, un nouveau enrollment_tok en est généré avec un nouveau lien.	VALIDE

3.3. Espace Apprenti (WebAuthn et Consultation)

ID	Pré-requis	Description de l'action	Résultat Attendu	Statut
APP-01	Smartphone connecté au LAN	Scanner le QR Code généré par le formateur.	Ouverture de la page /enroll?token=.. avec le nom de l'apprenti et le bouton d'action.	VALIDE
APP-02	Page d'enrôlement ouverte	Cliquer sur "Créer ma clé d'accès".	L'API generate_registration_options est appelée. L'invite FaceID/TouchID s'affiche nativement.	VALIDE
APP-03	Validation Biométrique	Valider l'empreinte digitale.	La signature est validée par Flask, le token est détruit en BDD, l'utilisateur est redirigé vers /docs.	VALIDE
APP-04	Apprenti non connecté	Accéder à /login, saisir le nom, cliquer sur "Se connecter".	L'invite biométrique s'affiche. Après validation, création de la variable de session et accès autorisé.	VALIDE

ID	Pré-requis	Description de l'action	Résultat Attendu	Statut
APP-05	Apprenti connecté	Vérification des accès dans /docs.	La requête SQL avec les clauses LEFT JOIN filtre correctement pour n'afficher que les modules autorisés (via utilisateur ou classe).	VALIDE
APP-06	Apprenti connecté	Cliquer sur le bouton "Aperçu du module" sur une carte.	Une modale s'ouvre contenant la description courte, le contenu riche (commentaires/ vidéos formatés via Summernote) et le lien vers le PDF technique.	VALIDE

4. Bilan Global et Conclusion

Le projet atteint l'intégralité des objectifs fixés par le cahier des charges initial. La transition d'une architecture simple vers une infrastructure Docker micro-services a permis de fiabiliser le déploiement sur les Bancs KLX.

Le point d'orgue du projet est l'implémentation réussie de la technologie WebAuthn (Passkeys) en réseau local fermé. La résolution des problèmes de sécurité stricts imposés par les navigateurs modernes (notamment la validation du certificat SSL Let's Encrypt et la concordance stricte du RP_ID) démontre un niveau de sécurité d'entreprise.

L'interface, refondue en Bootstrap 5, offre aux apprentis et aux formateurs une expérience utilisateur fluide, centralisée et sans mot de passe, réduisant ainsi drastiquement les frictions lors des sessions de travaux pratiques.